

Guide to the "Python for Neuroscience" Notebook

M1 Neurosciences 2026–2027 — Introduction to Programming

Nicolas Catz

This guide describes the notebook `Scripts/Python_intro_neuroscience_EN.ipynb`, which is used during the "Introduction to programming" session of the refresher week of the M1 student from AMU. It explains what the notebook covers, how it is organised, and how to work through it — read this before you open the notebook itself.

If you have not yet installed VSCode, Python, and the Jupyter extension, do that first — see `tutorial_setup_vscode_python_git.md`.

1. What is this notebook?

`Python_intro_neuroscience_EN.ipynb` is a **Jupyter notebook**: a document that mixes explanations (text), runnable Python code, and the results of that code (numbers, tables, plots) all in one file. It assumes **you have never programmed before**, and every new Python concept is immediately illustrated with a real neuroscience example — membrane potentials, ion concentrations, action potentials, etc.

By the end of the notebook, you will be able to:

- Write basic Python code (variables, calculations, text, lists, loops, conditions, functions)
- Use NumPy and Matplotlib, the two core scientific libraries for numerical computing and plotting
- Calculate and plot the **Nernst equilibrium potential** and the **Goldman-Hodgkin-Katz (GHK) resting potential** — the same physiology you see in the electrophysiology lecture, but reproduced with your own code

How to open it

1. Open VSCode.
2. **File** → **Open Folder...** and select the project folder containing the `Scripts` folder.
3. In the file explorer on the left, click `Scripts/Python_intro_neuroscience_EN.ipynb`.
4. VSCode will open it as a notebook (thanks to the Jupyter extension). You may be asked to select a **kernel** (the Python interpreter that runs the code) — pick the Python 3 installation you set up earlier.

How to work through it

- The notebook is made of **cells**. Markdown cells (like this text) contain explanations; code cells contain Python.
- Click a code cell and press **Shift + Enter** to run it — the result appears directly underneath.
- Run the cells **in order**, from top to bottom: later cells often reuse variables and functions defined earlier.
- Read the `#` comments inside the code — they explain what each line does.
- **Exercise cells** contain `...` (an ellipsis) where you need to write your own code. Running them before completing them will raise an error — that is expected, it just means it's your turn to fill in the blanks.

2. Structure of the notebook

The notebook has 10 sections, each building on the previous one:

#	Section	Python concepts	Neuroscience example
1	Variables	storing values (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>)	resting potential, number of neurons, ion name
2	Calculations	arithmetic operators, <code>numpy.log</code> , <code>numpy.exp</code>	Nernst equation computed step by step
3	Text	<code>print()</code> , f-strings, formatting numbers	displaying <code>E_K = -89.0 mV</code> cleanly
4	Lists	creating, indexing, slicing, <code>.append()</code>	list of brain regions, list of ion potentials
5	NumPy	arrays, <code>np.linspace</code> , vectorised operations, masks	computing <code>E_K</code> for a whole range of concentrations at once
6	Conditions	<code>if</code> / <code>elif</code> / <code>else</code> , comparison & logical operators	classifying a neuron as at rest / depolarised / hyperpolarised
7	Loops	<code>for</code> , <code>while</code> , <code>zip()</code> , accumulating results	computing <code>E_ion</code> for several ions in one loop
8	Functions	<code>def</code> , parameters, default values, <code>return</code> , docstrings	writing reusable <code>nernst()</code> and <code>goldman()</code> functions
9	Plots	Matplotlib: <code>plt.subplots</code> , styling, annotations, subplots	plotting <code>E_K</code> vs. <code>[K⁺]</code> extracellular
10	Final project	combining everything above	building a full explorer of the resting membrane potential

Each section follows the same pattern: a short explanation, one or two worked code examples, then an **exercise** for you to complete on your own.

3. The physiology running through the notebook

To keep the programming concepts grounded in something concrete, almost every example revolves around the same core neuroscience formula, the **Nernst equation**, which gives the equilibrium potential of an ion:

$$E_{ion} = \frac{RT}{zF} \ln \frac{[ion]_{ext}}{[ion]_{int}}$$

- `R` = gas constant, `T` = temperature (Kelvin), `F` = Faraday constant, `z` = ion valence
- `[ion]_ext` / `[ion]_int` = extracellular / intracellular concentration

Section 2 computes this by hand; Section 8 wraps it into a reusable `nernst()` function; Section 9 plots it; and Section 10 combines it with the **Goldman-Hodgkin-Katz (GHK) equation** (which accounts for several ions at once, weighted by their membrane permeability) to explore how the resting membrane potential changes

with extracellular K^+ concentration and temperature — directly connecting back to the electrophysiology session earlier in the week.

4. Exercises and the final project

- Exercises are short and appear after each section (e.g. Exercise 1.1, 2.1, ...). They ask you to reuse the concept just introduced with new numbers (e.g. compute `E_Na` instead of `E_K`).
- Two cells (Exercises 7.1 and 8.1) intentionally show an error output in the notebook — this is the "before" state, left as a working example of what happens when the `...` placeholders haven't been filled in yet. Replace the `...` with real code and re-run the cell to fix it.
- **Section 10 — Final project** ties everything together: you will reuse your `nernst()` and `goldman()` functions to study how hyperkalaemia (high extracellular K^+) and temperature affect neuronal excitability, ending with a "final challenge" function (`explore_GHK`) that reproduces the shift from resting potential to action potential peak.

Don't worry about getting every exercise perfect — the goal is to practice the mechanics of writing and running Python code with a meaningful, worked example.

5. After the notebook

The summary table at the end of the notebook recaps every concept learned. The suggested next step is the **"HH Fil rouge" notebook**, which uses these same skills to simulate a real action potential with the Hodgkin-Huxley model.

If you get stuck, contact:

- Dr. Nicolas CATZ — nicolas.catz@univ-amu.fr